

Documentación Técnica del Código de Control Manual WASD para Unitree R1

Robotics 4.0

Información general del código

Robot objetivo: Unitree R1

Lenguaje: C++

Función principal: Teleoperación local del robot mediante teclado en terminal Linux

Interfaz de operación: Teclas WASD, Q/E, espacio y X

API utilizada: Unitree R1 Loco API y Unitree R1 Loco Client

Tipo de documento: Documentación técnica específica del código

Documento técnico de análisis, operación y validación

30 de junio de 2026

Índice

1. Propósito del documento	2
2. Objetivo del código	2
3. Descripción general de la implementación	2
3.1. Componentes funcionales	2
4. Dependencias y entorno de ejecución	2
4.1. Librerías estándar	3
4.2. Librerías Unitree	3
5. Parámetros configurables	3
6. Captura inmediata de teclado	3
6.1. Lógica aplicada	3
7. Inicialización del robot	4
7.1. Secuencia de arranque	4
8. Mapa de comandos de teclado	4
8.1. Convención de ejes	4
9. Ciclo principal de operación	5
9.1. Flujo lógico	5
9.2. Manejo de teclas no válidas	5
10. Consideraciones de seguridad operacional	5
10.1. Condiciones recomendadas de prueba	6
11. Compilación y ejecución	6
11.1. Procedimiento general	6
12. Validación técnica del código	6
12.1. Pruebas recomendadas	7
13. Limitaciones del código	7
14. Conclusión	7
Anexo A. Código fuente documentado	9

1. Propósito del documento

Este documento describe el funcionamiento técnico de un código en C++ diseñado para controlar manualmente el robot Unitree R1 mediante pulsaciones directas de teclado en una terminal Linux. La implementación permite enviar comandos de locomoción al robot usando la API de Unitree, sin requerir una interfaz gráfica ni un sistema externo de teleoperación.

El alcance de esta documentación se limita al código suministrado: inicialización de comunicación, captura inmediata de teclado, asociación de teclas con comandos de movimiento, parada del robot y salida controlada del programa.

2. Objetivo del código

Implementar un mecanismo de teleoperación local para el robot Unitree R1, donde el operador pueda controlar desplazamiento lineal, desplazamiento lateral, giro sobre el eje vertical, frenado y cierre del programa mediante teclas predefinidas.

Resultado funcional esperado

El programa debe iniciar la conexión con el robot, activar el cliente de locomoción, permanecer a la espera de una tecla y ejecutar el movimiento correspondiente hasta que el operador envíe una nueva orden, active el freno o cierre el programa.

3. Descripción general de la implementación

El código se estructura en dos bloques funcionales. El primero corresponde a la función `getch()`, encargada de modificar temporalmente el comportamiento de la terminal para capturar una tecla sin necesidad de presionar *Enter*. El segundo corresponde a la función `main()`, donde se inicializa la comunicación con el robot, se configura el cliente de locomoción y se ejecuta el ciclo de lectura de comandos.

3.1. Componentes funcionales

Componente	Función dentro del código
<code>getch()</code>	Captura una pulsación de teclado de forma inmediata, sin esperar salto de línea y sin mostrar la tecla en pantalla.
<code>ChannelFactory::Init()</code>	Inicializa el canal de comunicación del SDK de Unitree usando la interfaz de red configurada.
<code>LocoClient</code>	Cliente de locomoción utilizado para iniciar el modo de control, enviar velocidades y detener el movimiento.
<code>client.Move()</code>	Envía velocidades lineales y angulares al robot en función de la tecla presionada.
<code>client.StopMove()</code>	Ejecuta la orden de parada de movimiento. Se usa para frenar y antes de salir del programa.
<code>switch(key)</code>	Asocia cada tecla válida con una acción de movimiento, parada o salida.

4. Dependencias y entorno de ejecución

4.1. Librerías estándar

- `chrono`: permite generar pausas temporales controladas.
- `iostream`: permite imprimir mensajes de estado en terminal.
- `thread`: permite suspender la ejecución durante el inicio del robot.
- `termios.h`: permite modificar el modo de lectura de la terminal Linux.
- `unistd.h`: permite usar `read()` para capturar caracteres desde la entrada estándar.

4.2. Librerías Unitree

- `unitree/robot/r1/loco/r1_loco_api.hpp`
- `unitree/robot/r1/loco/r1_loco_client.hpp`

Estas cabeceras permiten acceder a la API de locomoción del robot R1 y crear el cliente usado para enviar comandos de movimiento.

Condición de ejecución

El programa depende de una interfaz de red válida para comunicarse con el robot. En el código se usa `eth10`; si la máquina de control utiliza otra interfaz, este valor debe modificarse antes de compilar o ejecutar.

5. Parámetros configurables

El código define tres parámetros principales que deben revisarse antes de la operación física.

Parámetro	Valor actual	Descripción técnica
<code>network_interface</code>	<code>eth10</code>	Nombre de la interfaz de red usada por el SDK para comunicarse con el robot. Debe coincidir con la interfaz real del equipo de control.
<code>speed</code>	<code>0.3f</code>	Velocidad lineal usada para avance, retroceso y desplazamiento lateral. Su unidad operativa es metros por segundo.
<code>turn_speed</code>	<code>0.5f</code>	Velocidad angular usada para giros a izquierda y derecha. Su unidad operativa es radianes por segundo.

6. Captura inmediata de teclado

La función `getch()` modifica temporalmente la configuración de la terminal para que cada tecla sea capturada de forma directa. Por defecto, una terminal Linux trabaja en modo canónico, lo que significa que acumula la entrada hasta que el usuario presiona *Enter*. Para control robótico, ese comportamiento no es adecuado, porque introduce una demora innecesaria entre la intención del operador y la respuesta del robot.

6.1. Lógica aplicada

1. Se guarda la configuración actual de la terminal mediante `tcgetattr()`.

2. Se desactiva `ICANON` para evitar el buffer de línea.
3. Se desactiva `ECHO` para ocultar la tecla presionada.
4. Se configura `VMIN = 1`, de modo que la lectura espere exactamente un carácter.
5. Se captura la tecla con `read()`.
6. Se restaura el modo canónico y el eco de la terminal.
7. Se retorna el carácter capturado.

Implicación operativa

El programa permanece bloqueado esperando una tecla. Cada pulsación genera una orden puntual de movimiento, parada o salida. El robot no recibe comandos continuos por temporizador; recibe una nueva orden únicamente cuando el operador presiona una tecla válida.

7. Inicialización del robot

La función `main()` inicia definiendo la interfaz de red, inicializando el canal de comunicación del SDK y creando un cliente de locomoción.

7.1. Secuencia de arranque

1. Definición de la interfaz de red mediante `network_interface`.
2. Inicialización de `ChannelFactory` con el dominio 0 y la interfaz seleccionada.
3. Creación de `unitree::robot::r1::LocoClient`.
4. Inicialización del cliente mediante `client.Init()`.
5. Configuración de tiempo máximo de espera con `client.SetTimeout(10.f)`.
6. Inicio del robot mediante `client.Start()`.
7. Pausa de un segundo para permitir estabilización inicial antes de recibir comandos.

Criterio de arranque correcto

El arranque se considera correcto cuando el programa imprime el menú de control y el robot queda disponible para recibir comandos de movimiento desde teclado.

8. Mapa de comandos de teclado

El control manual se basa en un esquema WASD ampliado con teclas de giro, frenado y salida.

8.1. Convención de ejes

La estructura de `client.Move(x, y, yaw)` permite separar el movimiento en tres componentes: velocidad longitudinal, velocidad lateral y velocidad angular. En este código, el eje longitudinal se controla con W/S, el eje lateral con A/D y el giro sobre el eje vertical con Q/E.

Tecla	Acción	Comando enviado
W	Avance	<code>client.Move(speed, 0.0f, 0.0f)</code>
S	Retroceso	<code>client.Move(-speed, 0.0f, 0.0f)</code>
A	Desplazamiento lateral izquierdo	<code>client.Move(0.0f, speed, 0.0f)</code>
D	Desplazamiento lateral derecho	<code>client.Move(0.0f, -speed, 0.0f)</code>
Q	Giro hacia la izquierda	<code>client.Move(0.0f, 0.0f, turn_speed)</code>
E	Giro hacia la derecha	<code>client.Move(0.0f, 0.0f, -turn_speed)</code>
Espacio	Freno operativo	<code>client.StopMove()</code>
X	Salida del programa	<code>client.StopMove()</code> y cierre del ciclo

9. Ciclo principal de operación

El ciclo principal se ejecuta mientras la variable `running` sea verdadera. En cada iteración, el programa espera una tecla, evalúa su valor mediante una estructura `switch` y ejecuta la acción asociada.

9.1. Flujo lógico

Secuencia de control

1. El programa espera una pulsación mediante `getch()`.
2. La tecla se compara contra los casos válidos del `switch`.
3. Si la tecla corresponde a movimiento, se envía `client.Move()`.
4. Si la tecla es espacio, se ejecuta `client.StopMove()`.
5. Si la tecla es `x`, se detiene el robot y se termina el ciclo.
6. Si la tecla no está definida, el programa la ignora.

9.2. Manejo de teclas no válidas

El bloque `default` no ejecuta ninguna acción. Esta decisión evita que pulsaciones accidentales envíen comandos no previstos al robot. Desde el punto de vista operativo, el comportamiento es seguro porque el programa solo responde ante teclas explícitamente definidas.

10. Consideraciones de seguridad operacional

El código incluye dos mecanismos básicos de seguridad: una tecla de parada y una salida controlada. La tecla de espacio ejecuta `StopMove()` sin terminar el programa, mientras que la tecla `x` detiene el movimiento antes de cerrar el ciclo principal.

Punto crítico

El operador debe tener acceso inmediato a la tecla de espacio durante cualquier prueba física. La parada por teclado es el mecanismo de frenado operativo del programa y debe verificarse antes de ejecutar movimientos prolongados.

10.1. Condiciones recomendadas de prueba

- Ejecutar primero con velocidades bajas si el espacio es reducido.
- Verificar que la interfaz de red corresponda al enlace real con el robot.
- Probar `StopMove()` antes de realizar desplazamientos.
- Mantener el robot en un área libre de obstáculos durante las primeras pruebas.
- Tener un operador atento al comportamiento físico del robot durante toda la ejecución.

11. Compilación y ejecución

La compilación depende de la estructura del workspace y de la ubicación del SDK de Unitree. El archivo debe compilarse enlazando correctamente las librerías requeridas por `r1_loco_client.hpp` y `r1_loco_api.hpp`.

11.1. Procedimiento general

1. Confirmar la interfaz de red conectada al robot.
2. Ajustar el valor de `network_interface` si no corresponde a `eth10`.
3. Compilar el código dentro del entorno donde estén disponibles las cabeceras y librerías del SDK.
4. Ejecutar el binario desde una terminal Linux.
5. Verificar que aparezca el menú de control.
6. Probar primero la tecla de parada.
7. Ejecutar movimientos individuales y cerrar con `X`.

Nota de implementación

Si el código se integra a un proyecto con CMake, el objetivo de compilación debe incluir el archivo fuente y enlazar las dependencias del SDK de Unitree. El nombre del ejecutable puede definirse según la estructura interna del workspace.

12. Validación técnica del código

La validación debe confirmar que el programa cumple su función específica: controlar manualmente el robot mediante teclado, detenerlo de forma segura y cerrar sin dejar movimiento activo.

12.1. Pruebas recomendadas

PT-01. Validación de conexión con el robot

Objetivo: confirmar que el cliente de locomoción inicializa correctamente usando la interfaz de red configurada.

Resultado esperado: el programa inicia, muestra el menú de control y queda a la espera de comandos.

PT-02. Validación de comandos lineales

Objetivo: verificar que las teclas w y s generen avance y retroceso respectivamente.

Resultado esperado: el robot responde en la dirección correspondiente y se detiene al presionar espacio.

PT-03. Validación de desplazamiento lateral

Objetivo: comprobar que las teclas A y D ejecuten desplazamientos laterales diferenciados.

Resultado esperado: el robot realiza strafe hacia izquierda o derecha según la tecla presionada.

PT-04. Validación de giro

Objetivo: verificar que las teclas Q y E ejecuten giro sobre el eje vertical.

Resultado esperado: el robot gira hacia el lado correspondiente sin desplazamiento lineal.

PT-05. Validación de parada y salida

Objetivo: comprobar que la tecla espacio detenga el movimiento y que la tecla X cierre el programa después de enviar una orden de parada.

Resultado esperado: no queda movimiento activo después de frenar o cerrar el programa.

13. Limitaciones del código

- El programa no implementa control proporcional ni rampa de aceleración.
- El movimiento depende de la última orden enviada hasta que se ejecute una nueva orden o una parada.
- No existe lectura de estado del robot dentro del ciclo principal.
- No se implementa evasión de obstáculos ni validación del entorno.
- El control requiere una terminal activa y acceso directo al teclado.
- La interfaz de red está definida de forma fija en el código.

14. Conclusión

El código documentado implementa una herramienta directa y funcional para teleoperar el robot Unitree R1 desde una terminal Linux. Su lógica es simple, verificable y adecuada

para pruebas iniciales de locomoción, validación de comandos básicos y control manual en entornos supervisados.

La implementación se apoya en tres elementos centrales: captura inmediata de teclado, cliente de locomoción de Unitree y asociación explícita entre teclas y velocidades. La correcta operación física depende de una interfaz de red válida, parámetros de velocidad adecuados y verificación previa del comando de parada.



Anexo A. Código fuente documentado

```

1  #include <chrono>
2  #include <iostream>
3  #include <thread>
4  #include <termios.h>
5  #include <unistd.h>
6
7  #include "unitree/robot/r1/loco/r1_loco_api.hpp"
8  #include "unitree/robot/r1/loco/r1_loco_client.hpp"
9
10 // Captura una tecla sin requerir Enter.
11 char getch() {
12     char buf = 0;
13     struct termios old = {0};
14
15     if (tcgetattr(0, &old) < 0) perror("tcsetattr()");
16
17     old.c_lflag &= ~ICANON;
18     old.c_lflag &= ~ECHO;
19     old.c_cc[VMIN] = 1;
20     old.c_cc[VTIME] = 0;
21
22     if (tcsetattr(0, TCSANOW, &old) < 0) perror("tcsetattr ICANON");
23     if (read(0, &buf, 1) < 0) perror("read()");
24
25     old.c_lflag |= ICANON;
26     old.c_lflag |= ECHO;
27
28     if (tcsetattr(0, TCSADRAIN, &old) < 0) perror("tcsetattr ~ICANON");
29
30     return buf;
31 }
32
33 int main() {
34     std::string network_interface = "eth10";
35     unitree::robot::ChannelFactory::Instance()->Init(0, network_interface);
36
37     unitree::robot::r1::LocoClient client;
38     client.Init();
39     client.SetTimeout(10.f);
40
41     float speed = 0.3f;
42     float turn_speed = 0.5f;
43
44     std::cout << "Iniciando robot..." << std::endl;
45     client.Start();
46     std::this_thread::sleep_for(std::chrono::seconds(1));
47
48     std::cout << "\n=== CONTROL POR TECLADO ACTIVADO ===" << std::endl;
49     std::cout << " W : Adelante           Q : Girar Izquierda" << std::endl;
50     std::cout << " S : Atras             E : Girar Derecha" << std::endl;
51     std::cout << " A : Izquierda (Strafe)" << std::endl;
52     std::cout << " D : Derecha (Strafe)" << std::endl;
53     std::cout << " [ESPACIO] : DETENER (Freno)" << std::endl;
54     std::cout << " X : Salir y apagar" << std::endl;
55     std::cout << "=====\\n" << std::endl;
56
57     bool running = true;

```

```
58
59 while (running) {
60     char key = getch();
61
62     switch (key) {
63         case 'w': case 'W':
64             std::cout << "Adelante" << std::endl;
65             client.Move(speed, 0.0f, 0.0f);
66             break;
67
68         case 's': case 'S':
69             std::cout << "Atras" << std::endl;
70             client.Move(-speed, 0.0f, 0.0f);
71             break;
72
73         case 'a': case 'A':
74             std::cout << "Izquierda" << std::endl;
75             client.Move(0.0f, speed, 0.0f);
76             break;
77
78         case 'd': case 'D':
79             std::cout << "Derecha" << std::endl;
80             client.Move(0.0f, -speed, 0.0f);
81             break;
82
83         case 'q': case 'Q':
84             std::cout << "Girar Izquierda" << std::endl;
85             client.Move(0.0f, 0.0f, turn_speed);
86             break;
87
88         case 'e': case 'E':
89             std::cout << "Girar Derecha" << std::endl;
90             client.Move(0.0f, 0.0f, -turn_speed);
91             break;
92
93         case ' ':
94             std::cout << "DETENIDO" << std::endl;
95             client.StopMove();
96             break;
97
98         case 'x': case 'X':
99             std::cout << "Saliendo del programa..." << std::endl;
100             client.StopMove();
101             running = false;
102             break;
103
104         default:
105             break;
106     }
107 }
108
109 return 0;
110 }
```